

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA KHOA HỌC MÁY TÍNH

HỌC PHẦN: PHÂN TÍCH KHAI PHÁ DỮ LIỆU LỚN



BÁO CÁO BÀI TẬP LỚN NHÓM 9

**ĐỀ TÀI : HỆ THỐNG PHÁT HIỆN ĐẠO VĂN TRÊN TẬP DỮ
LIỆU QUY MÔ LỚN**

Họ và tên

Mã sinh viên

Nguyễn Trường Giang

B22DCKH034

Phạm Quốc Anh

B22DCKH006

Trương Quốc Bình

B22DCKH008

Giảng viên hướng dẫn: GV. Đặng Hoàng Long

Hà Nội - 2025

GIỚI THIỆU DỰ ÁN VÀ BỘ DỮ LIỆU	4
1. TỔNG QUAN DỰ ÁN	4
1.1. Mục tiêu	4
1.2. Phân loại các cấp độ đạo văn	5
2. PHẠM VI DỮ LIỆU VÀ HẠ TẦNG KỸ THUẬT.	6
KIẾN TRÚC DATA LAKE (MEDALLION ARCHITECTURE)	7
3. THU THẬP VÀ XỬ LÝ DỮ LIỆU	8
3.1. Giai đoạn 1: Thu thập & Chuẩn hóa Metadata (Bronze)	8
3.2. Giai đoạn 2: Cào PDF	10
3.3. Giai đoạn 3: Trích xuất Text từ PDF (Bronze → Silver)	10
4. QUY TRÌNH XỬ LÝ VÀ LÀM SẠCH VĂN BẢN	11
5. XÂY DỰNG SILVER+ DATASET	13
5.1. Join 3 nguồn dữ liệu	13
5.2. Tách section tự động	13
5.3. Schema Silver+	13
CÁC KỸ THUẬT PHÁT HIỆN ĐẠO VĂN	15
6. HỆ THỐNG MINHASH LSH (PHÁT HIỆN TƯƠNG ĐỒNG CÚ PHÁP)	15
Bài toán phát hiện đạo văn trên dữ liệu lớn	15
6.1. Tổng quan MinHash và LSH	16
Tạo MinHash Signatures	17
6.2. Xây dựng LSH Index (Datasketch)	19
7. HỆ THỐNG FUZZY MATCHING (	20
8. HỆ THỐNG FAISS SEMANTIC INDEX (PHÁT HIỆN TƯƠNG ĐỒNG NGỮ NGHĨA)	21
8.1. Tổng quan	21
8.2. Chunking (Chia đoạn văn)	22
8.3. Embedding Model: SPECTER	22
8.4. FAISS Index	23
9. ENGINE PHÁT HIỆN ĐẠO VĂN 5 LỚP	23
9.1. Luồng xử lý tổng thể	24
9.2. Chi tiết 5 lớp phân tích	24
KẾT QUẢ ĐẠT ĐƯỢC	25
Kịch bản thực nghiệm	25
Đánh giá hệ thống	27
Phân tích và Đánh giá kỹ thuật	27
Hạn chế hiện tại và Định hướng phát triển	28

GIỚI THIỆU DỰ ÁN VÀ BỘ DỮ LIỆU

1. TỔNG QUAN DỰ ÁN

1.1. Mục tiêu

Dự án hướng tới xây dựng một hệ sinh thái Big Data hoàn chỉnh, có khả năng phát hiện các hành vi đạo văn học thuật trên tập dữ liệu ArXiv — kho lưu trữ tri thức với hơn 3 triệu bài báo khoa học. Hệ thống được thiết kế để tiếp nhận tài liệu PDF mới, thực hiện đối sánh với toàn bộ kho dữ liệu đã được lập chỉ mục nhằm đưa ra phân tích định lượng về mức độ tương đồng và xác suất đạo văn.

***Hiểu đúng về các cấp độ đạo văn**

Trước khi lên kế hoạch, cần hiểu rõ kẻ thù là gì. Đạo văn trong bài báo khoa học có 5 cấp độ từ thô đến tinh:

Cấp 1 — Copy thô (Verbatim)

"The proposed method achieves 94.3% accuracy on ImageNet dataset"

→ "The proposed method achieves 94.3% accuracy on ImageNet dataset"

Dễ phát hiện nhất. Jaccard similarity rất cao.

Cấp 2 — Đổi từ đồng nghĩa (Synonym Substitution)

"The proposed method achieves 94.3% accuracy"

→ "The suggested approach attains 94.3% precision"

Jaccard thấp nhưng Semantic cao.

Cấp 3 — Đảo cấu trúc câu (Paraphrase)

"We trained the model on 50,000 images for 100 epochs"

→ "100 training epochs were conducted using a dataset of 50,000 images"

Jaccard rất thấp, Semantic vẫn cao.

Cấp 4 — Đạo văn ý tưởng (Idea Plagiarism)

Không copy câu chữ, nhưng lấy toàn bộ phương pháp,

đóng góp, thí nghiệm của người khác mà không trích dẫn.

Không thể phát hiện bằng text comparison thuần túy.

Cấp 5 — Tự đạo văn (Self-Plagiarism)

Tác giả tái sử dụng bài báo cũ của chính mình

mà không công bố — vi phạm quy định xuất bản.

Cần so sánh cùng tác giả với nhau.

1.2. Phân loại các cấp độ đạo văn

Để tối ưu hóa khả năng nhận diện, nhóm thực hiện phân loại hành vi đạo văn thành 5 cấp độ từ cơ bản đến phức tạp:

Cấp độ	Hình thức	Đặc điểm kỹ thuật	Giải pháp đối soát
1	Copy nguyên văn (Verbatim)	Chỉ số Jaccard Similarity rất cao.	MinHash + LSH
2	Thay thế từ đồng	Jaccard thấp	Fuzzy Matching

Cấp độ	Hình thức	Đặc điểm kỹ thuật	Giải pháp đối soát
	nghĩa	nhưng Semantic Similarity cao.	
3	Thay đổi cấu trúc câu (Paraphrase)	Cấu trúc văn bản thay đổi, ý nghĩa bảo toàn.	FAISS Semantic Search
4	Đạo văn ý tưởng	Lấy cắp phương pháp, thí nghiệm không trích dẫn.	Keyword/Numeric Matching
5	Tự đạo văn (Self-Plagiarism)	Tác giả tái sử dụng nội dung cũ chưa công bố.	Author Metadata Check

2. PHẠM VI DỮ LIỆU VÀ HẠ TẦNG KỸ THUẬT.

Thành phần	Chi tiết
Nguồn dữ liệu	ArXiv metadata
Kích thước tập dữ liệu	~4 GB metadata JSONL + ~438GB file pdf
Số lượng bài báo	Hơn 300000 bài
Ngôn ngữ	Tiếng Anh
Lĩnh vực	CS, Physics, Math, Biology, Economics...
Hạ tầng lưu trữ	Google Cloud Storage (bucket: bigdata-n9-ptit-final)
Nền tảng xử lý	Google Colab + Dataproc Cluster (GCP) + VMs
Project GCP	bigdataptit2026

KIẾN TRÚC DATA LAKE (MEDALLION ARCHITECTURE)

Hệ thống áp dụng mô hình Data Lake 3 tầng nhằm tối ưu hóa tính truy vết và hiệu quả phân tích:

Tầng	Đường dẫn GCS	Nội dung	Định dạng
Bronze	gs://bigdata-n9-ptit-final/bronze/	Dữ liệu thô nguyên bản	JSONL, PDF
Bronze - Metadata	bronze/raw_metadata/arxiv/2026_04_30/	Metadata chuẩn hóa từ Kaggle	JSONL (flat)

Bronze - PDFs	bronze/raw_pdfs/arxiv/<date>/	File PDF gốc cào từ arxiv.org	PDF binary
Silver - Text	silver/arxiv_text_parquet/	Text thô trích xuất từ PDF	Parquet
Silver - Cleaned	silver/arxiv_cleaned_parquet/	Text đã làm sạch 9 bộ lọc	Parquet
Silver+	silver/arxiv_silver_plus/	Dataset đầy đủ sau join PySpark	Parquet
Gold- Chunks	gold/chunks_parquet/	Đoạn văn (chunks) theo câu	Parquet
Gold - Signatures	gold/minhash_signatures/	MinHash signatures 128 permutations	Parquet
gold - LSH Index	gold/minhash_index/	LSH index + MinHash objects	Pickle, JSON
gold -FAISS	gold/faiss_index/	FAISS vector index + chunk metadata	Binary, Parquet

3. THU THẬP VÀ XỬ LÝ DỮ LIỆU

3.1. Giai đoạn 1: Thu thập & Chuẩn hóa Metadata (Bronze)

Metadata ArXiv được cào từ trang web chính thức của ArXiv, dữ liệu được lưu dưới dạng JSON

Sơ đồ luồng xử lý:

→ Stream Processing (dòng theo dòng) → Chuẩn hóa JSON flat

→ Lưu JSONL → gcloud storage cp → GCS Bronze

Schema metadata chuẩn hóa:

Trường	Kiểu dữ liệu	Mô tả
doc_id	String	ID định danh duy nhất (arxiv_<id>)
hash_sha256	String	Mã băm SHA-256 của source_url
title	String	Tiêu đề bài báo (đã làm sạch ký tự xuống dòng)
abstract	String	Tóm tắt bài báo
authors	Array<String>	Danh sách tác giả (Firstname Lastname)
categories	Array<String>	Danh mục ArXiv (cs.AI, math.ST...)
publish_date	String	Ngày cập nhật (update_date)
doi	String	DOI
source_url	String	URL bài báo trên arxiv.org
pdf_url	String	URL tải PDF trực tiếp
local_path_bronze	String	Đường dẫn GCS lưu PDF
scraped_at	ISO8601	Timestamp thu thập dữ liệu

3.2. Giai đoạn 2: Cào PDF

PDF được cào từ arxiv.org và đẩy thẳng lên GCS (streaming upload), tránh chiếm dung lượng disk cục bộ.

- HTTP Session với retry 5 lần và exponential backoff
- Kiểm tra trùng lặp (resume): bỏ qua file đã tồn tại trên GCS
- Rate limiting: delay ngẫu nhiên 1.5–2.5 giây giữa các request
- Phân công công việc theo dải chỉ số (START_INDEX → END_INDEX) để chạy song song nhiều Colab

```
PS D:\DOWNLOADS\full script bigdata\files> & C:/Users/
Đang quét dung lượng thư mục: bronze/raw_pdfs/arxiv/
-----
TỔNG KẾT:
Số lượng file: 306,404 PDFs
Tổng dung lượng: 438.84 GB
```

3.3. Giai đoạn 3: Trích xuất Text từ PDF (Bronze → Silver)

Sử dụng PyMuPDF (fitz) trích xuất text từ PDF với chiến lược batch download tối ưu hóa cho dữ liệu 300GB+:

- Download batch ~10.000 file về local (/tmp/pdfs_batch/) để tránh tràn disk
- Trích xuất song song bằng ProcessPoolExecutor
- Checkpoint theo file_path: không xử lý lại file đã hoàn thành khi bị crash
- Dọn dẹp disk tự động sau mỗi batch
- Ghi kết quả thành Parquet batch 5.000 file/part, đẩy trực tiếp lên GCS
- Lọc file rác: bỏ qua tài liệu có word_count ≤ 10
- Dữ liệu được lưu vào silver/arxiv_text_parquet/

Buckets > bigdata-n9-ptit-final > silver > arxiv_text_parquet						
Create folder Upload Transfer data Create batch operations New Other services						
Filter by name prefix only Filter Filter objects and folders Show Live objects only						
☐	Name	Size	Type	Created	Storage class	Last modif
☐	part-00000.parquet	115.7 MB	application/octet-stream	May 6, 2026, 12:18:36 AM	Standard	May 6, 2026
☐	part-00001.parquet	114.9 MB	application/octet-stream	May 6, 2026, 12:18:53 AM	Standard	May 6, 2026
☐	part-00002.parquet	109.8 MB	application/octet-stream	May 6, 2026, 12:19:36 AM	Standard	May 6, 2026
☐	part-00003.parquet	113.2 MB	application/octet-stream	May 6, 2026, 12:19:53 AM	Standard	May 6, 2026
☐	part-00004.parquet	120.3 MB	application/octet-stream	May 6, 2026, 12:20:38 AM	Standard	May 6, 2026
☐	part-00005.parquet	115 MB	application/octet-stream	May 6, 2026, 12:20:56 AM	Standard	May 6, 2026
☐	part-00006.parquet	118.1 MB	application/octet-stream	May 6, 2026, 12:21:44 AM	Standard	May 6, 2026
☐	part-00007.parquet	120.8 MB	application/octet-stream	May 6, 2026, 12:22:04 AM	Standard	May 6, 2026
☐	part-00008.parquet	125 MB	application/octet-stream	May 6, 2026, 12:23:01 AM	Standard	May 6, 2026
☐	part-00009.parquet	122.8 MB	application/octet-stream	May 6, 2026, 12:23:22 AM	Standard	May 6, 2026
☐	part-00010.parquet	126.2 MB	application/octet-stream	May 6, 2026, 12:24:27 AM	Standard	May 6, 2026
☐	part-00011.parquet	131.9 MB	application/octet-stream	May 6, 2026, 12:24:50 AM	Standard	May 6, 2026
☐	part-00012.parquet	141.8 MB	application/octet-stream	May 6, 2026, 12:26:14 AM	Standard	May 6, 2026
☐	part-00013.parquet	141.3 MB	application/octet-stream	May 6, 2026, 12:26:38 AM	Standard	May 6, 2026
☐	part-00014.parquet	139.5 MB	application/octet-stream	May 6, 2026, 12:28:07 AM	Standard	May 6, 2026
☐	part-00015.parquet	142.4 MB	application/octet-stream	May 6, 2026, 12:28:32 AM	Standard	May 6, 2026

4. QUY TRÌNH XỬ LÝ VÀ LÀM SẠCH VĂN BẢN

Đầu tiên, chúng ta sẽ cần trích xuất các thực thể cần xử lí trong 1 file PDF, chỉ giữ lại những thứ cần thiết để phân tích đạo văn.

Quy trình gồm 9 tầng xử lý tuần tự:

Tầng	Tên xử lý	Mô tả chi tiết
1	ArXiv Header Removal	Xóa header định danh ArXiv (arXiv:xxxx.xxx [cs.AI] DD Mon YYYY)
2	Journal Metadata Removal	Xóa thông tin template tạp chí (WSPC, Preprint submitted to, Draft version...)
3	References Extraction	Tìm và cắt phần References/Bibliography trong 40% cuối bài, đánh dấu has_references
4	Page Number	Xóa số trang đứng độc lập (\n 123 \n, page

	Removal	X of Y)
5	Ligature Fix	Sửa ligature Unicode (ff, fi, fl, ffi, ffl, st)
6	Unicode Normalization	Chuẩn hóa NFKC - thống nhất các ký tự tương đương
7	Control Character Removal	Xóa ký tự không in được (0x00-0x1F, 0x7F)
8	Hyphenation Fix	Nối lại từ bị tách xuống dòng (word-\nword → word)
9	Email/URL/Whitespace	Xóa email, URL HTTP(S); chuẩn hóa khoảng trắng, dòng trống thừa

Quality Flag phân loại chất lượng sau làm sạch:

- ok: $\text{word_count_clean} \geq 500$ và $\text{reduction_ratio} \geq 0.3$
- low_quality: $300 \leq \text{word_count_clean} < 500$
- empty: $\text{word_count_clean} < 300$
- over_cleaned: $\text{reduction_ratio} < 0.3$ (mất quá nhiều nội dung)

Data sau khi làm sạch được lưu tại đây: `silver/arxiv_cleaned_parquet/`

Buckets > bigdata-n9-ptit-final > silver > arxiv_cleaned_parquet						
Create folder Upload Transfer data Create batch operations New Other services						
Filter by name prefix only Filter Filter objects and folders Show Live objects only						
☐	Name	Size	Type	Created	Storage class	Last modified
☐	part-00000.parquet	104.2 MB	application/octet-stream	May 6, 2026, 4:47:28 PM	Standard	May 6, 2026
☐	part-00001.parquet	103.8 MB	application/octet-stream	May 6, 2026, 4:48:10 PM	Standard	May 6, 2026
☐	part-00002.parquet	98.2 MB	application/octet-stream	May 6, 2026, 4:48:50 PM	Standard	May 6, 2026
☐	part-00003.parquet	101.8 MB	application/octet-stream	May 6, 2026, 4:49:31 PM	Standard	May 6, 2026
☐	part-00004.parquet	107.9 MB	application/octet-stream	May 6, 2026, 4:50:14 PM	Standard	May 6, 2026
☐	part-00005.parquet	103.4 MB	application/octet-stream	May 6, 2026, 4:50:57 PM	Standard	May 6, 2026
☐	part-00006.parquet	106.5 MB	application/octet-stream	May 6, 2026, 4:51:41 PM	Standard	May 6, 2026
☐	part-00007.parquet	108.1 MB	application/octet-stream	May 6, 2026, 4:52:25 PM	Standard	May 6, 2026
☐	part-00008.parquet	111.7 MB	application/octet-stream	May 6, 2026, 4:53:10 PM	Standard	May 6, 2026

5. XÂY DỰNG SILVER+ DATASET

Bước này thực hiện việc join 3 nguồn dữ liệu riêng biệt và tạo dataset thống nhất làm nền tảng cho tất cả các bước phân tích tiếp theo.

5.1. Join 3 nguồn dữ liệu

Nguồn	Key join	Nội dung
silver/arxiv_text_parquet/	file_path	Text thô (raw_text)
silver/arxiv_cleaned_parquet/	file_path (left join)	Text đã làm sạch + quality flags
bronze/raw_metadata/	doc_id (left join)	Metadata: title, authors, categories, abstract

5.2. Tách section tự động

PySpark UDF phân tích cấu trúc bài báo để tách introduction và body:

- Nhận diện phần Introduction bằng regex pattern (1. Introduction, I. Introduction, INTRODUCTION)
- Nhận diện section tiếp theo: số đề mục (2., II.), hoặc tên section chuẩn (RELATED WORK, METHODOLOGY, CONCLUSION...)
- Trả về 3 trường: introduction, body, và clean_text toàn bộ

5.3. Schema Silver+

Cột	Kiểu	Nguồn
paper_id	String	Tên file PDF (filename)

file_path	String	Đường dẫn GCS đầy đủ
raw_text	String	Text thô từ PyMuPDF
clean_text	String	Text sau 9 tầng làm sạch
abstract	String	Tóm tắt (từ metadata)
introduction	String	Phần mở đầu (tách tự động)
body	String	Phần nội dung chính
word_count_raw	Integer	Số từ text thô
word_count_clean	Integer	Số từ sau làm sạch
title	String	Tiêu đề (từ metadata)
authors	Array<String>	Danh sách tác giả
categories	Array<String>	Danh mục ArXiv
quality_flag	String	ok / low_quality / empty / over_cleaned

Data được lưu dưới dạng Parquet trong thư mục silver/arxiv_silver_plus/ với khoảng ~50GB dữ liệu

Buckets > bigdata-n9-ptit-final > silver > arxiv_silver_plus							
Create folder Upload Transfer data Create batch operations New Other services							
Filter by name prefix only				Filter Filter objects and folders		Show Live objects only	
☐	Name	Size	Type	Created	Storage class	Last modified	
☐	part-00299-272e1150-c553-446b...	63.2 MB	application/octet-stream	May 6, 2026, 5:50:36 PM	Standard	May 6, 2026,	
☐	part-00300-272e1150-c553-446b...	59.9 MB	application/octet-stream	May 6, 2026, 5:50:35 PM	Standard	May 6, 2026,	
☐	part-00301-272e1150-c553-446b...	58.4 MB	application/octet-stream	May 6, 2026, 5:50:36 PM	Standard	May 6, 2026,	
☐	part-00302-272e1150-c553-446b...	63.2 MB	application/octet-stream	May 6, 2026, 5:50:36 PM	Standard	May 6, 2026,	
☐	part-00303-272e1150-c553-446b...	60.5 MB	application/octet-stream	May 6, 2026, 5:50:02 PM	Standard	May 6, 2026,	
☐	part-00304-272e1150-c553-446b...	60.1 MB	application/octet-stream	May 6, 2026, 5:50:36 PM	Standard	May 6, 2026,	
☐	part-00305-272e1150-c553-446b...	60.9 MB	application/octet-stream	May 6, 2026, 5:50:37 PM	Standard	May 6, 2026,	
☐	part-00306-272e1150-c553-446b...	61.2 MB	application/octet-stream	May 6, 2026, 5:50:36 PM	Standard	May 6, 2026,	
☐	part-00307-272e1150-c553-446b...	58.6 MB	application/octet-stream	May 6, 2026, 5:50:36 PM	Standard	May 6, 2026,	
☐	part-00308-272e1150-c553-446b...	63.5 MB	application/octet-stream	May 6, 2026, 5:50:02 PM	Standard	May 6, 2026,	
☐	part-00309-272e1150-c553-446b...	60.7 MB	application/octet-stream	May 6, 2026, 5:50:37 PM	Standard	May 6, 2026,	
☐	part-00310-272e1150-c553-446b...	59.6 MB	application/octet-stream	May 6, 2026, 5:50:37 PM	Standard	May 6, 2026,	

CÁC KỸ THUẬT PHÁT HIỆN ĐẠO VĂN

6. HỆ THỐNG MINHASH LSH (PHÁT HIỆN TƯƠNG ĐỒNG CÚ PHÁP)

Bài toán phát hiện đạo văn trên dữ liệu lớn

Phát hiện đạo văn (Plagiarism Detection) là bài toán xác định mức độ tương đồng giữa các tài liệu nhằm phát hiện hành vi sao chép nội dung, diễn đạt lại ý tưởng hoặc tái sử dụng tài liệu mà không trích dẫn nguồn.

Trong môi trường học thuật hiện đại, số lượng bài báo khoa học tăng rất nhanh, đặc biệt trên kho dữ liệu ArXiv với hàng triệu tài liệu PDF. Điều này khiến việc so sánh tuần tự từng cặp văn bản trở nên bất khả thi do độ phức tạp tính toán quá lớn.

Nếu có N tài liệu, việc so sánh brute-force cần:

$$O(N^2)$$

Điều này không phù hợp với hệ thống Big Data.

Do đó, hệ thống cần các kỹ thuật:

- Giảm số lượng candidate cần so sánh
- Tìm kiếm gần đúng (Approximate Search)
- Lập chỉ mục hiệu quả
- So khớp cả cú pháp và ngữ nghĩa

Nhóm sử dụng kết hợp:

- MinHash
- Locality Sensitive Hashing (LSH)
- Fuzzy String Matching
- FAISS Semantic Search

để xây dựng hệ thống phát hiện đạo văn đa tầng.

6.1. Tổng quan MinHash và LSH

Khái niệm MinHash: MinHash (Min-wise Independent Permutations) là một kỹ thuật xác suất giúp ước lượng nhanh chóng độ tương đồng Jaccard (Jaccard Similarity) giữa hai tập hợp dữ liệu lớn mà không cần phải so sánh trực tiếp từng phần tử. Thuật toán hoạt động bằng cách băm (hash) các đoạn văn bản ngắn (n-gram/shingle) qua nhiều hàm băm khác nhau để tạo ra một "chữ ký" (signature) đại diện cho văn bản đó.

Gọi A và B là tập hợp các n-gram (shingles) của hai văn bản. Độ tương đồng Jaccard (Jaccard Similarity) giữa hai văn bản được tính bằng tỷ lệ giữa phần giao và phần hợp của hai tập hợp:

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

Thuật toán MinHash băm các phần tử qua nhiều hàm băm ngẫu nhiên. Đặc tính

toán học cốt lõi của MinHash là xác suất để giá trị băm nhỏ nhất của hai tập hợp bằng nhau chính bằng độ tương đồng Jaccard của chúng:

$$P(h_{\min} A = h_{\min} B) = J(A, B)$$

Sai số xấp xỉ của ước lượng này tỷ lệ nghịch với căn bậc hai của số lượng phép băm (NUM_PERM), được tính theo công thức:

$$\text{Sai số} \approx \frac{1}{\sqrt{\text{NUM_PERM}}}$$

(Do đó, với NUM_PERM = 128, sai số ước lượng rơi vào khoảng +/- 0,088 , đủ đáp ứng yêu cầu cho bước lọc thô của hệ thống).

Khái niệm LSH: Locality Sensitive Hashing (LSH) là một kỹ thuật phân cụm (hashing) đặc biệt. Khác với các hàm băm thông thường cố gắng tránh đụng độ (collision), LSH cố ý xếp các phần tử có độ tương đồng cao vào cùng một "bucket" (thùng).

Tác dụng trong phát hiện đạo văn: MinHash và LSH kết hợp tạo thành Lớp bảo vệ thứ 1 (L1) trong hệ thống, chuyên xử lý các hành vi đạo văn Cấp độ 1 (Copy nguyên văn). Khi có một bài báo mới đưa vào, LSH cho phép hệ thống truy vấn với thời gian $O(1)$ thay vì phải quét qua toàn bộ 3 triệu bài báo. Nó đóng vai trò "lọc thô", tìm ra một danh sách ứng viên (candidate list) có độ tương đồng cú pháp vượt ngưỡng (Jaccard $\geq 0,3$) để các lớp phân tích sâu hơn tiếp quản.

Tạo MinHash Signatures

Tham số	Giá trị	Ý nghĩa
SHINGLE_K	5	Kích thước shingle (5-gram từ)
NUM_PERM	128	Số lượng hash permutation
LARGE_PRIME	2,147,483,647	Số nguyên tố lớn cho hash universal
Hash function	MD5 mod prime	Tạo integer hash cho mỗi shingle
Broadcast	HASH_A, HASH_B (seed=42)	Tham số hash cố định, broadcast toàn cluster

Với mỗi bài báo: tạo tập shingle hash → áp dụng 128 phép biến đổi min-hash → signature vector [128] kiểu Integer.

SHINGLE_K = 5 (5-gram từ)

- K nhỏ (2-3): quá nhiều shingle trùng ngẫu nhiên → false positive cao
- K lớn (7-10): quá ít shingle trùng → bỏ sót paraphrase nhẹ
- K=5 là chuẩn trong plagiarism detection, cân bằng giữa precision và recall.

Với bài báo khoa học ~5000 từ, 5-gram đủ dài để mang ngữ nghĩa ("lattice Boltzmann method for") nhưng đủ ngắn để bắt được copy-paste có sửa nhỏ.

NUM_PERM = 128

- Sai số ước lượng Jaccard $\approx 1/\sqrt{\text{NUM_PERM}}$
- 128 → sai số $\approx 8.8\%$, đủ chính xác cho bước lọc thô
- 64 → sai số 12.5%, hơi thô
- 256 → sai số 6.25%, chính xác hơn nhưng tốn gấp đôi RAM/disk
- 128 là trade-off phổ biến giữa chính xác và tài nguyên

LARGE_PRIME = 2,147,483,647 ($2^{31} - 1$)

- Số nguyên tố Mersenne lớn nhất trong phạm vi 32-bit integer
- Hash function: $h(x) = (a \cdot x + b) \bmod p$, cần p là số nguyên tố để phân phối

đều

- $2^{31}-1$ đủ lớn để tránh collision, đủ nhỏ để tính toán nhanh trên integer 64-bit

MD5 mod prime

- MD5 biến shingle string $\rightarrow$ integer đều (128-bit $\rightarrow$ mod prime $\rightarrow$ 31-bit)
- Nhanh hơn SHA256, không cần cryptographic security ở đây
- Chỉ cần phân phối đều, MD5 đáp ứng tốt

seed=42 cho HASH_A, HASH_B

- Cố định seed đảm bảo reproducibility: build index và query dùng cùng hash params
- 42 là convention phổ biến, giá trị cụ thể không ảnh hưởng chất lượng
- Nếu seed khác nhau giữa build và query $\rightarrow$ MinHash không khớp $\rightarrow L1 = 0$ (bug đã gặp trước đó)

Kết quả:

```
Paper: arxiv_0704.0397.pdf
Type: <class 'datasketch.minhash.MinHash'>
Num perm: 128
Signature (10 gia tri dau): [ 26680 957066 615634 641355 318998 227973 583147 359482 1142213 216109]
Min/Max: 5377 / 2814177
Dtype: uint64

Paper: arxiv_0704.0855.pdf
Type: <class 'datasketch.minhash.MinHash'>
Num perm: 128
Signature (10 gia tri dau): [ 135257 604720 51688 800859 62044 1477246 1398544 929266 374699 1026117]
Min/Max: 5908 / 1983751
Dtype: uint64

Paper: arxiv_0704.1301.pdf
Type: <class 'datasketch.minhash.MinHash'>
Num perm: 128
Signature (10 gia tri dau): [ 674741 834873 557318 20133 255068 6207 69727 80274 346573 1199339]
Min/Max: 1883 / 1505176
Dtype: uint64
```

6.2. Xây dựng LSH Index (Datasketch)

Datasketch MinHashLSH được sử dụng để build index, hỗ trợ query realtime O(1):

- Threshold: 0.5 (Jaccard similarity tối thiểu để xét là candidate)
- Lưu 2 file: lsh_index.pkl (index cấu trúc), minhashes.pkl (tồn bộ MinHash objects)

```
=== LSH INDEX ===  
Type: <class 'datasketch.lsh.MinHashLSH'>  
Threshold: 128  
Num bands (b): 25  
Num rows per band (r): 25  
Num perm: 3200
```

7. HỆ THỐNG FUZZY MATCHING (

- Khái niệm: Fuzzy Matching (So khớp mờ) thường dựa trên khoảng cách Levenshtein – thuật toán đo lường khoảng cách chỉnh sửa (edit distance) giữa hai chuỗi ký tự. Khoảng cách này được tính bằng số bước tối thiểu (thêm, sửa, hoặc xóa ký tự) để biến đổi chuỗi này thành chuỗi kia.
- Khoảng cách Levenshtein giữa hai chuỗi a và b (kí hiệu là $Levenshtein(a,b)$) là số thao tác chỉnh sửa tối thiểu để biến đổi chuỗi này thành chuỗi kia. Để đánh giá mức độ giống nhau theo tỷ lệ phần trăm (phục vụ việc đối sánh với ngưỡng 85%), hệ thống tính toán Tỷ lệ tương đồng (Levenshtein Ratio) theo công thức:

$$Ratio = 1 - \frac{Levenshtein(a,b)}{\max(|a|, |b|)}$$

- Tác dụng trong phát hiện đạo văn:

- Được sử dụng làm Lớp bảo vệ thứ 2 (L2) để đối phó với đạo văn Cấp độ 2 (Thay thế từ đồng nghĩa, sửa đổi nhỏ lẻ).
- Trong khi LSH làm việc ở cấp độ toàn bài báo, Fuzzy Matching tiến hành soi chiếu ở cấp độ vi mô (từng câu). Hệ thống sẽ tách bài báo thành các câu và tính tỷ lệ Levenshtein với các câu của bài ứng viên (hơn 85%).

8. HỆ THỐNG FAISS SEMANTIC INDEX (PHÁT HIỆN TƯƠNG ĐỒNG NGỮ NGHĨA)

8.1. Tổng quan

Facebook AI Similarity Search (FAISS) là một thư viện tối ưu hóa cao độ cho việc tìm kiếm độ tương đồng và phân cụm các vector (embeddings) trong không gian đa chiều, đặc biệt hiệu quả với các tập dữ liệu cực lớn.

Khái niệm Semantic Embedding: Là việc sử dụng các mô hình ngôn ngữ lớn (ở dự án này là Transformers BERT-based SPECTER) để chuyển đổi văn bản thành các vector toán học (768 chiều). Các văn bản có ý nghĩa giống nhau sẽ nằm gần nhau trong không gian vector này, dù chúng dùng từ vựng hoàn toàn khác nhau.

Tác dụng trong phát hiện đạo văn: Đây là vũ khí cốt lõi của Lớp bảo vệ thứ 3 (L3), nhắm tới hành vi đạo văn Cấp độ 3 và 4 (Paraphrase cấu trúc câu, ẩn cấp ý tưởng). Thay vì đếm từ trùng lặp, FAISS tính toán khoảng cách Cosine (Cosine Similarity) giữa các vector ngữ nghĩa. Điều này giúp hệ thống phát hiện ra những đoạn văn đã được viết lại hoàn toàn tinh vi nhưng vẫn mang luồng tư tưởng, phương pháp giống hệt bài gốc.

$$\text{Cosine}(u, v) = \frac{u \cdot v}{||u|| \times ||v||}$$

Giá trị này biểu diễn góc giữa hai vector. Giá trị càng tiến gần về 1 (góc càng hẹp), hai đoạn văn bản càng có ý nghĩa tương đồng nhau. Điều này giúp hệ thống bắt được những đoạn văn đã bị viết lại hoàn toàn tinh vi nhưng vẫn mang luồng tư tưởng, phương pháp giống hệt bài gốc.

8.2. Chunking (Chia đoạn văn)

Mỗi bài báo được chia thành các chunk nhỏ hơn để tăng độ chính xác so sánh:

Tham số	Giá trị	Ý nghĩa
CHUNK_SIZE	4	Số câu tối đa mỗi chunk
Câu tối thiểu	≥ 20 ký tự	Lọc câu quá ngắn
Chunk tối thiểu	≥ 50 ký tự	Lọc chunk rỗng/vô nghĩa
Section	abstract, introduction, body	Chunk được gán nhãn section nguồn
chunk_uid	paper_id + chunk_id	Định danh duy nhất toàn hệ thống

8.3. Embedding Model: SPECTER

Mô hình allenai/specter (Transformers) được chọn để tạo embedding chuyên biệt cho văn bản khoa học:

- Kiến trúc: BERT-based, được fine-tune trên tập dữ liệu trích dẫn học thuật
- Output dimension: 768 chiều (CLS token của last hidden state)
- Max length: 512 tokens (truncation)

- Batch size: 128
- Normalize L2 trước khi nạp vào FAISS để sử dụng Inner Product = Cosine Similarity

8.4. FAISS Index

Điều kiện	Loại Index	Đặc điểm
$n < 100.000$ vectors	IndexFlatIP	Exact search, Inner Product metric
$n \geq 100.000$ vectors	IndexIVFFlat($nlist=\sqrt{n}$, max 4096)	Approximate search, phân cụm IVF

Kết quả:

Buckets > bigdata-n9-ptit-final > intermediate > faiss_index							
Create folder Upload Transfer data Create batch operations New Other services							
Filter by name prefix only		Filter objects and folders				Show Live objects only	
☐	Name	Size	Type	Created	Storage class	Last modified	
☐	chunk_metadata.parquet	176.7 MB	application/octet-stream	May 7, 2026, 4:43:04 PM	Standard	May 7, 2026	Download More
☐	faiss_index.bin	1.8 GB	application/octet-stream	May 7, 2026, 4:42:50 PM	Standard	May 7, 2026	Download More
☐	index_config.json	114 B	application/json	May 7, 2026, 4:43:07 PM	Standard	May 7, 2026	Download More

9. ENGINE PHÁT HIỆN ĐẠO VĂN 5 LỚP

Module chính của hệ thống. Khi nhận PDF đầu vào, hệ thống xử lý qua 5 lớp phân tích độc lập rồi tổng hợp điểm cuối cùng.

9.1. Luồng xử lý tổng thể

Bước	Mô tả
1	Extract + Clean: Trích xuất text từ PDF → Làm sạch 9 tầng → Tách abstract
2	L1 MinHash: Tính MinHash signature → Query LSH index → Tìm candidate
3	L3 Semantic: Embed abstract bằng SPECTER → Search FAISS top-20
4	L2 Fuzzy: Levenshtein ratio trên từng câu với các candidate từ L1+L3
5	L4 Concept: So khớp keyword, số liệu, claim giữa các bài
6	L5 Self-Plagiarism: Kiểm tra tác giả trùng với candidate
7	Scoring: Tổng hợp điểm có trọng số → Phán quyết

9.2. Chi tiết 5 lớp phân tích

Lớp 1 (L1): MinHash LSH - Tương đồng cú pháp

- Tạo MinHash signature cho toàn bộ text bằng CÙNG tham số với pipeline offline
- Query datasketch LSH index → trả về candidate list
- Tính Jaccard similarity thực với từng candidate qua hashvalues

Ngưỡng lọc: Jaccard ≥ 0.3 mới đưa vào candidate set

Lớp 2 (L2): Fuzzy String Matching - Tương đồng cú pháp câu

- Tách bài báo thành danh sách câu (tối đa 200 câu, ưu tiên câu dài)
- So sánh từng câu với câu trong bài candidate bằng Levenshtein ratio

- Bỏ qua cặp câu có độ lệch độ dài > 2.5 lần (tối ưu hiệu năng)
- Ngưỡng: FUZZY_THRESHOLD = 0.85 ($\geq 85\%$ giống nhau)
- Kết quả: fuzzy_count (số câu giống) và fuzzy_ratio

Lớp 3 (L3): Semantic Search - Tương đồng ngữ nghĩa

- Embed abstract bằng SPECTER $\rightarrow$ normalize L2 $\rightarrow$ FAISS search top-20
- Điểm cosine similarity được chuẩn hóa về $[0,1]$: $(\cos - 0.85) / 0.15$
- Phát hiện paraphrase: bài báo có ý nghĩa tương tự nhưng diễn đạt khác

Lớp 4 (L4): Concept Matching - Tương đồng khái niệm

- So sánh title similarity (SequenceMatcher ratio)
- Keyword overlap: tập từ dài ≥ 4 ký tự trong abstract (Jaccard)
- Claim overlap: các cụm từ đặc trưng (we propose, we present, novel, our method...)
- Numeric overlap: so khớp các số liệu % trong abstract
- Công thức: $0.3 \times \text{title} + 0.3 \times \text{keyword} + 0.2 \times \text{claim} + 0.2 \times \text{numeric}$

Lớp 5 (L5): Self-Plagiarism Detection

- Kiểm tra tác giả đầu vào có trùng với tác giả bài candidate
- Phát hiện tự đạo văn: tác giả tái sử dụng nội dung từ bài trước của chính mình
- Đầu vào: danh sách tên tác giả do người dùng cung cấp

KẾT QUẢ ĐẠT ĐƯỢC

Kịch bản thực nghiệm

Để đánh giá hiệu năng và độ chính xác của hệ thống, nhóm tiến hành trích xuất ngẫu nhiên một bài báo khoa học đã được lưu trữ sẵn trong hệ thống (bronze/raw_pdfs/) để làm dữ liệu đầu vào (query document). Bài báo được thử nghiệm có tiêu đề: "Sparsity-certifying Graph Decompositions", chứa 200 câu sau quá trình tiền xử lý.

```
[1/7] Extract + clean (9 tang)...
Title: Sparsity-certifying Graph Decompositions
So cau: 200

[2/7] L1 - MinHash LSH...
1 candidates | score=0.9922 | 0.4s
arxiv_0704.0002.pdf | J=0.9922 | Sparsity-certifying Graph Decompositions

[3/7] L3 - Semantic (FAISS)...
1 candidates | 0.6s
arxiv_0704.0002.pdf | cos=0.9117 | Sparsity-certifying Graph Decompositions

Union: 21 (L1:1 + L3:1)

[4/7] L2 - Fuzzy match...
1 matches | score=1.0 | 2.2s
arxiv_0704.0002.pdf | fuzzy=1.0 (100/200)

[5/7] L4 - Concept match...
1 matches | score=0.6 | 6.9s

[6/7] L5 - Self-plagiarism...
0 matches | score=0.0

[7/7] KET QUA
=====
Verdict:      PLAGIARISM
Diem tong:    100.0%
Do tin cay:   HIGH
Self-plag:    Khong
L1=99.2% L2=100.0% L3=91.1% L4=60.0% L5=0.0%
Thoi gian:   10.2s

-----
DANH SACH BAI TUONG TU
-----

[1] RAT CAO | Sparsity-certifying Graph Decompositions
L1=99% L2=100% L3=0% Max=100%
https://arxiv.org/abs/0704.0002
```

Đánh giá hệ thống

Hệ thống hoàn thành toàn bộ quy trình phân tích 5 lớp (bao gồm cả trích xuất, làm sạch và query index) chỉ trong thời gian 10.2 giây cho một tài liệu. Kết quả chẩn đoán chi tiết qua các lớp đối sánh như sau:

- Lớp 1 (MinHash LSH - Cú pháp): Trích xuất được 1 ứng viên tiềm năng (arxiv_0704.0002.pdf) với chỉ số Jaccard ước lượng đạt 99.2%.
- Lớp 3 (FAISS Semantic - Ngữ nghĩa): Trích xuất cùng ứng viên trên với khoảng cách ngữ nghĩa (Cosine Similarity) đạt 91.1%.
- Lớp 2 (Fuzzy Matching - Vi mô): Đối chiếu chéo 200/200 câu văn đều có sự trùng khớp, tỷ lệ Levenshtein đạt 100%.
- Lớp 4 (Concept Matching): Điểm tương đồng khái niệm đạt 60.0%.
- Lớp 5 (Self-Plagiarism): Không phát hiện dấu hiệu tự đạo văn (0.0%).

Phán quyết cuối cùng của hệ thống: Tỷ lệ đạo văn 100.0% với độ tin cậy CAO (HIGH). Dữ liệu đối chiếu gốc: <https://arxiv.org/abs/0704.0002>.

Phân tích và Đánh giá kỹ thuật

Quá trình thực nghiệm cho thấy hệ thống hoạt động chính xác và đạt tốc độ truy xuất thời gian thực rất ấn tượng nhờ kiến trúc LSH và FAISS. Dù thử nghiệm bằng chính bài báo gốc, kết quả trả về giữa các lớp có sự phân hóa phản ánh đúng bản chất toán học của từng thuật toán:

- Về chỉ số L1 (MinHash đạt 99.2% thay vì 100%): Thuật toán MinHash với cấu hình 128 permutations cung cấp giá trị xấp xỉ của Jaccard Similarity chứ không phải giá trị tuyệt đối. Sai số vi phân 0.8 hoàn toàn nằm trong biên độ sai số lý thuyết cho phép của hệ thống (xấp xỉ 8,8%). Bên cạnh đó, các sai số về tokenization hoặc seed của hàm băm trong quá trình build index và query cũng đóng góp vào sự chênh lệch nhỏ này.
- Về chỉ số L3 (FAISS đạt 91.1%): SPECTER là một mạng nơ-ron đánh giá

cụm ngữ nghĩa bao hàm cả context xung quanh đoạn văn bản (chunk). Do đó, hai văn bản dù nội dung giống hệt nhau nhưng nếu bị cắt chunk với các context viền khác nhau thì vector embedding trong không gian 768 chiều sẽ tịnh tiến lệch nhau một biên độ nhỏ, dẫn tới Cosine Similarity < 1.0 . Thuật toán này chứng minh hệ thống có khả năng nhận diện tốt các ý tưởng tương đồng mà không bị phụ thuộc vào câu chữ chính xác (như kết quả tuyệt đối 100\% của L2 Fuzzy Matching).

Hạn chế hiện tại và Định hướng phát triển

Dù đã xây dựng thành công pipeline 5 lớp bảo vệ, hệ thống vẫn còn một số điểm cần tối ưu trong các nghiên cứu tương lai:

1. Lớp 4 (Concept Matching): Cần tiếp tục nghiên cứu, tinh chỉnh bộ trọng số và tối ưu hóa hàm mục tiêu để thuật toán đánh giá chính xác hơn mức độ vay mượn các tham số kỹ thuật và "claim" của bài báo.
2. Lớp 5 (Self-Plagiarism): Module phát hiện tự đạo văn hiện tại còn thô sơ. Hệ thống cần tích hợp thêm thuật toán Chuẩn hóa Thực thể Tác giả (Author Entity Resolution) để giải quyết triệt để các trường hợp tác giả đổi tên, dùng tên viết tắt hoặc sai lệch metadata.